

Tecnatura Superior en Desarrollo de Software

Módulo 05 — Guards: validación de precondiciones

Actividad Práctica — CU-CLI-001: Alta de Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

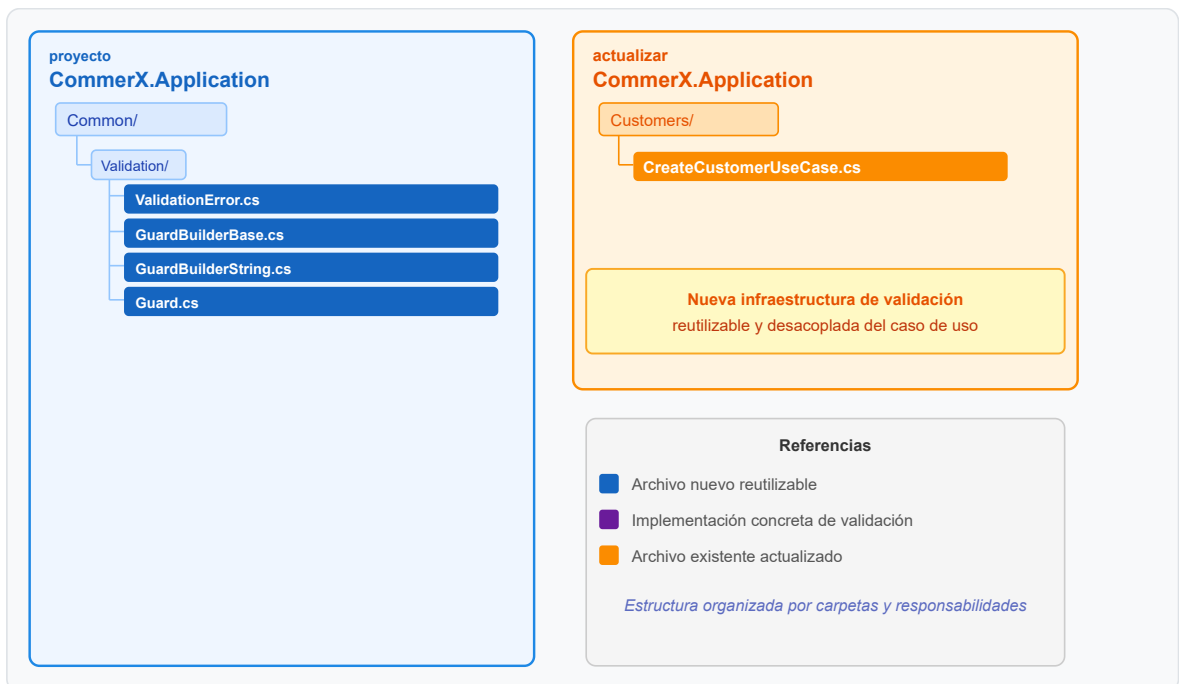
Ejemplo práctico

En el **Módulo 04** implementaste `CreateCustomerUseCase` con sus puertos e interfaces. En ese momento el UseCase solo capturaba `DomainException` si el dominio rechazaba un dato.

En esta actividad agregás la capa de validación de precondiciones técnicas: los **Guards**. Antes de que el dominio sea invocado, el UseCase verificará que los campos del DTO llegaron bien formados y devolverá **todos los errores juntos** si alguno falla.

Objetivo: Crear los helpers de validación (`ValidationError` , `GuardBuilderBase` , `GuardBuilderString` , `Guard`), definir el contrato `IModelValidatorHub<TModel>` , implementar `CreateCustomerValidatorHub` , actualizar `CreateCustomerUseCase` para inyectar el hub y registrar las nuevas dependencias en `DependencyContainer` .

Archivos a crear o actualizar — estructura de proyectos



Archivo 1 — CommerX.Application / Common / Validation / Validation Error.cs

¿Qué hace esta clase?

Transporta la información de un error de validación: qué campo falló y cuál es el mensaje. No es una excepción — se acumula en una lista y se devuelve al llamador juntos con los demás errores encontrados. Usa primary constructor de C# 12 para mantenerla inmutable y sin lógica.

```

1  // CommerX.Application/Common/Validation/ValidationError.cs
2  namespace CommerX.Application.Common.Validation;
3
4  // primary constructor C# 12 - datos inmutables sin lógica
5  public class ValidationError(string propertyName, string message)
6  {
7      // nombre del campo - la UI lo usa para destacar el input incorrecto
8      public string PropertyName => propertyName;
9
10     // mensaje legible que se muestra al usuario
11     public string ErrorMessage => message;
12 }

```

Recordá: `ValidationError` es una clase simple — no un record ni una excepción. Los DTOs de este proyecto son clases; las excepciones se reservan para el dominio.

Archivo 2 — CommerX.Application / Common / Validation / GuardBuilderBase.cs

¿Qué hace esta clase?

Clase abstracta genérica que implementa el patrón Fluent Builder con CRTP (Curiously Recurring Template Pattern). Acumula los errores internamente y los expone como solo lectura. Cada subclase concreta pasa su propio tipo como argumento genérico para que el encadenamiento devuelva siempre el tipo concreto.

```
1 // CommerX.Application/Common/Validation/GuardBuilderBase.cs
2 namespace CommerX.Application.Common.Validation;
3
4 // CRTP: TBuilder es el tipo concreto que hereda esta clase
5 // el constraint where garantiza que el cast (TBuilder)this es seguro
6 public abstract class GuardBuilderBase<TBuilder>
7     where TBuilder : GuardBuilderBase<TBuilder>
8 {
9     // lista interna – protected para que las subclases puedan agregar errores
10    protected readonly List<ValidationError> _errors = new();
11
12    // nombre del campo que se está validando
13    protected readonly string _paramName;
14
15    // constructor protegido – solo las subclases lo invocan via base()
16    protected GuardBuilderBase(string paramName) => _paramName = paramName;
17
18    // expone los errores acumulados – el hub los combina después
19    public IReadOnlyList<ValidationError> Errors => _errors;
20
21    // true si no se registró ningún error
22    public bool IsValid => _errors.Count == 0;
23
24    // agrega un ValidationError – solo accesible para las subclases
25    protected void AddError(string message)
26        => _errors.Add(new ValidationError(_paramName, message));
27 }
```

Archivo 3 — CommerX.Application / Common / Validation / GuardBuilderString.cs

¿Qué hace esta clase?

Builder concreto para campos de texto. Hereda de `GuardBuilderInterface` — pasando su propio tipo como argumento genérico (CRTP) — para que cada método devuelva `GuardBuilderString` y el encadenamiento funcione sin perder el tipo. Expone cuatro reglas: `NotNullOrEmpty`, `MinLength`, `MaxLength` e `InvalidEmail`.

```
1 // CommerX.Application/Common/Validation/GuardBuilderString.cs
2 namespace CommerX.Application.Common.Validation;
3
4 // pasa su propio tipo al genérico – eso es el CRTP
5 public class GuardBuilderString : GuardBuilderInterface<GuardBuilderString>
6 {
7     // valor de texto que se está validando
8     private readonly string _value;
9
10    // llama al constructor base con el nombre del campo
11    public GuardBuilderString(string value, string paramName)
12        : base(paramName) => _value = value;
13
14    // verifica que el campo no sea nulo, vacío ni solo espacios
15    public GuardBuilderString NotNullOrEmpty(string? mensaje = null)
16    {
17        if (string.IsNullOrEmpty(_value))
18            AddError(mensaje ?? $"El campo '{_paramName}' es requerido.");
19        return this;
20    }
21
22    // verifica longitud mínima – se omite si el campo ya está vacío
23    public GuardBuilderString MinLength(int min, string? mensaje = null)
24    {
25        if (_value?.Length < min)
26            AddError(mensaje ?? $"'{_paramName}' debe tener al menos {min} caracteres.");
27        return this;
28    }
29
30    // verifica que no supere la longitud máxima permitida
31    public GuardBuilderString MaxLength(int max, string? mensaje = null)
32    {
33        if (_value?.Length > max)
34            AddError(mensaje ?? $"'{_paramName}' no puede superar {max} caracteres.");
35        return this;
36    }
37
38    // verifica formato básico de email – uso exclusivo para el campo Email
```

```
39     public GuardBuilderString InvalidEmail(string? mensaje = null)
40     {
41         var regex = new System.Text.RegularExpressions.Regex(
42             @"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$");
43         if (string.IsNullOrEmpty(_value) || !regex.IsMatch(_value))
44             AddError(mensaje ?? $"'{_paramName}' no tiene formato de correo valido.");
45         return this;
46     }
47 }
```

Archivo 4 — CommerX.Application / Common / Validation / Guard.cs

¿Qué hace esta clase?

Punto de entrada estático que crea el Builder correspondiente al tipo de dato recibido. El UseCase y el Hub interactúan solo con esta clase — nunca instancian los Builders directamente. Por ahora solo tiene la sobrecarga para `string`; en módulos posteriores se agregarán sobrecargas para `DateOnly` y `Guid`.

```
1 // CommerX.Application/Common/Validation/Guard.cs
2 namespace CommerX.Application.Common.Validation;
3
4 // clase estática — no se instancia, es el punto de entrada del Guard
5 public static class Guard
6 {
7     // Against — "defenderse contra" un valor de texto inválido
8     public static GuardBuilderString Against(string value, string paramName)
9         => new GuardBuilderString(value, paramName);
10 }
```

Archivo 5 — CommerX.Application / Common / Validation / IModelValidatorHub.cs

¿Qué hace esta interfaz?

Define el contrato que el UseCase usa para delegar toda la validación sin conocer qué Guards se aplican internamente. Un único método recibe el modelo y devuelve todos los errores encontrados — si la colección está vacía, el modelo es válido.

```
1 // CommerX.Application/Common/Validation/IModelValidatorHub.cs
2 namespace CommerX.Application.Common.Validation;
3
4 // puerto de validación — un método, una responsabilidad
5 public interface IModelValidatorHub<TModel>
6 {
7     // valida el modelo y devuelve todos los errores encontrados
```

```

8      // si la colección está vacía, el modelo es válido
9      IEnumerable<ValidationError> Validate(TModel model);
10     }

```

Archivo 6 — CommerX.Application / Customers / Validation / CreateCustomerValidatorHub.cs

¿Qué hace esta clase?

Implementación concreta del hub para `CreateCustomerRequest`. Aplica un Guard por cada campo del DTO, combina todos los errores y los devuelve como una sola secuencia. El UseCase no sabe qué reglas se aplican — solo recibe la lista resultante.

Campo	Reglas Guard
FullName	NotNullOrEmpty · MinLength(2) · MaxLength(100)
LastName	NotNullOrEmpty · MinLength(2) · MaxLength(100)
Document	NotNullOrEmpty · MinLength(7) · MaxLength(8)
Email	NotNullOrEmpty · InvalidEmail()
Phone	NotNullOrEmpty · MaxLength(20)
Address	NotNullOrEmpty · MaxLength(200)

```

1  // CommerX.Application/Customers/Validation/CreateCustomerValidatorHub.cs
2  using CommerX.Application.Common.Validation;
3
4  namespace CommerX.Application.Customers.Validation;
5
6  // hub concreto para CreateCustomerRequest
7  public class CreateCustomerValidatorHub
8      : IModelValidatorHub<CreateCustomerRequest>
9  {
10     public IEnumerable<ValidationError> Validate(CreateCustomerRequest request)
11     {
12         // un Guard por campo — cada uno acumula sus propios errores
13         var gFullName = Guard.Against(request.FullName, "FullName")
14             .NotNullOrEmpty().MinLength(2).MaxLength(100);
15
16         var gLastName = Guard.Against(request.LastName, "LastName")
17             .NotNullOrEmpty().MinLength(2).MaxLength(100);
18
19         var gDocument = Guard.Against(request.Document, "Document")
20             .NotNullOrEmpty().MinLength(7).MaxLength(8);
21

```

```
22     var gEmail = Guard.Against(request.Email, "Email")
23         .NotNullOrEmpty().InvalidEmail();
24
25     var gPhone = Guard.Against(request.Phone, "Phone")
26         .NotNullOrEmpty().MaxLength(20);
27
28     var gAddress = Guard.Against(request.Address, "Address")
29         .NotNullOrEmpty().MaxLength(200);
30
31     // combina todos los errores en una sola secuencia y la devuelve
32     return gFullName.Errors
33         .Concat(gLastName.Errors)
34         .Concat(gDocument.Errors)
35         .Concat(gEmail.Errors)
36         .Concat(gPhone.Errors)
37         .Concat(gAddress.Errors);
38 }
39 }
```

Archivo 7 — CommerX.Application / Customers / CreateCustomerUseCase.cs — actualizar

¿Qué cambia?

El UseCase recibe el hub por inyección de dependencias y lo invoca al inicio de `ExecuteAsync`. Si hay errores de precondition, notifica al `OutputPort` y retorna inmediatamente — sin invocar al dominio. Solo si el DTO es válido, continúa con la creación de la entidad.

```
1  // CommerX.Application/Customers/CreateCustomerUseCase.cs
2  using CommerX.Application.Common.Validation;
3  using CommerX.Domain.Common.Exceptions;
4
5  namespace CommerX.Application.Customers;
6
7  public class CreateCustomerUseCase : ICreateCustomerInputPort
8  {
9      private readonly ICreateCustomerOutputPort _outputPort;
10     private readonly ICustomerRepository _repository;
11     private readonly IModelValidatorHub<CreateCustomerRequest> _validator;
12
13     // el hub se inyecta igual que el repositorio y el outputPort
14     public CreateCustomerUseCase(
15         ICreateCustomerOutputPort outputPort,
16         ICustomerRepository repository,
17         IModelValidatorHub<CreateCustomerRequest> validator)
18     {
19         _outputPort = outputPort;
20         _repository = repository;
```

```
21     _validator = validator;
22 }
23
24 public async Task ExecuteAsync(CreateCustomerRequest request)
25 {
26     // PASO 1 – Guard Hub: condiciones técnicas del DTO
27     var errors = _validator.Validate(request).ToList();
28     if (errors.Count > 0)
29     {
30         // notifica todos los errores juntos – el dominio no es invocado
31         await _outputPort.ValidationErrorsAsync(errors);
32         return;
33     }
34
35     // PASO 2 – Dominio: construye la entidad con sus invariantes
36     try
37     {
38         // si algún dato viola una regla, Customer.Create lanza DomainException
39         var customer = Customer.Create(
40             request.FullName,
41             request.LastName,
42             request.Document,
43             request.Email,
44             request.Phone,
45             request.Address,
46             request.BirthDate);
47
48         // PASO 3 – Persistencia: agrega el cliente al repositorio
49         await _repository.AddAsync(customer);
50
51         // PASO 4 – Notificación: informa éxito al OutputPort
52         await _outputPort.HandleSuccessAsync(
53             new CreateCustomerResponse { CustomerId = customer.Id });
54     }
55     catch (DomainException ex)
56     {
57         // invariante de dominio violada – notifica el mensaje de la excepción
58         await _outputPort.HandleErrorAsync(ex.Message);
59     }
60 }
61 }
```

¿Por qué Guard antes de Customer.Create()?

El Guard verifica condiciones técnicas del DTO — que los campos llegaron y tienen formato mínimo aceptable. El dominio verifica invariantes de negocio — que la fecha de nacimiento implica mayoría de edad, que el documento tiene formato específico del país. Son dos capas distintas con responsabilidades distintas.

Archivo 8 — CommerX.Application / DependencyContainer.cs — actualizar

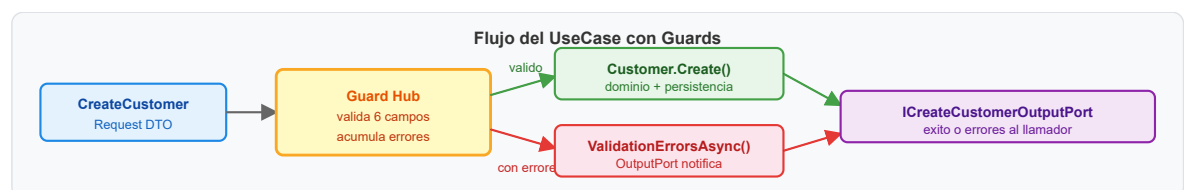
¿Qué cambia?

Se agrega el registro del hub como `Scoped` — mismo ciclo de vida que el `UseCase` y el `DbContext`. La inyección se realiza a través de la interfaz genérica `IModelValidatorHub<CreateCustomerRequest>`, no de la clase concreta.

```
1 // CommerX.Application/DependencyContainer.cs
2 using CommerX.Application.Common.Validation;
3 using CommerX.Application.Customers;
4 using CommerX.Application.Customers.Validation;
5
6 // el namespace truco evita agregar using en Program.cs
7 namespace Microsoft.Extensions.DependencyInjection;
8
9 public static class DependencyContainer
10 {
11     public static IServiceCollection AddCustomerUseCases(
12         this IServiceCollection services)
13     {
14         // UseCase – Scoped: una instancia por petición HTTP
15         services.AddScoped<ICreateCustomerInputPort, CreateCustomerUseCase>();
16
17         // hub – Scoped: mismo ciclo de vida que el UseCase
18         services.AddScoped<
19             IModelValidatorHub<CreateCustomerRequest>,
20             CreateCustomerValidatorHub>();
21
22         return services;
23     }
24 }
```

Explicación paso a paso

La implementación se realiza en cuatro etapas ordenadas. Cada etapa debe compilar correctamente antes de pasar a la siguiente.



Paso 1 — Crear los helpers de validación

Los archivos `ValidationError`, `GuardBuilderBase`, `GuardBuilderString` y `Guard` forman el núcleo reutilizable del sistema de validación. Deben existir antes que el hub porque el hub los usa. Todos viven en `Common/Validation/` — no son específicos de ningún caso de uso.

Paso 2 — Definir `IModelValidatorHub<TModel>`

La interfaz genérica desacopla el UseCase del hub concreto. El UseCase solo conoce `IModelValidatorHub<CreateCustomerRequest>` — no sabe qué Guards se aplican internamente. Si mañana se agregan o modifican reglas de precondition, el UseCase no necesita cambiar.

Paso 3 — Implementar `CreateCustomerValidatorHub`

El hub concreto aplica un Guard por campo y concatena todos los errores en una sola secuencia. El UseCase llama a `Validate()`, recibe la lista completa y la entrega al OutputPort de una sola vez — el usuario ve todos los errores juntos, no uno por vez.

Paso 4 — Actualizar UseCase y DependencyContainer

El UseCase agrega un tercer parámetro en el constructor. La validación ocurre antes del `try` que invoca al dominio: si hay errores, se notifica y se retorna. El DependencyContainer agrega un nuevo `AddScoped` registrando la interfaz genérica con la implementación concreta.

Resumen de ubicaciones

Archivo	Proyecto	Carpeta	Tipo
<code>ValidationError.cs</code>	CommerX.Application	Common/Validation/	class — DTO de error
<code>GuardBuilderBase.cs</code>	CommerX.Application	Common/Validation/	abstract class — CRTP base
<code>GuardBuilderString.cs</code>	CommerX.Application	Common/Validation/	class — builder concreto para string
<code>Guard.cs</code>	CommerX.Application	Common/Validation/	static class — punto de entrada

Archivo	Proyecto	Carpeta	Tipo
IModelValidatorHub.cs	CommerX.Application	Common/Validation/	interface — contrato genérico
CreateCustomerValidatorHub.cs	CommerX.Application	Customers/Validation/	class — implementación concreta
CreateCustomerUseCase.cs	CommerX.Application	Customers/	class — actualizar constructor y ExecuteAsync
DependencyContainer.cs	CommerX.Application	raíz del proyecto	static class — agregar registro del hub

Buenas prácticas y errores comunes

Guard vs. validación de dominio

ERROR COMUN

El Guard verifica reglas de negocio que pertenecen al dominio — como si el email tiene el formato específico de la empresa o si el documento corresponde a un tipo válido del país.

```
1 // MAL: regla de negocio en el Guard
2 // los dígitos del DNI son invariante del dominio, no precondition técnica
3 Guard.Against(request.Document, "Document")
4     .OnlyDigits(); // incorrecto — esa lógica pertenece al dominio
```

BUENA PRACTICA

El Guard verifica solo precondiciones técnicas: que el campo llegó, que tiene una longitud mínima aceptable. El dominio verifica las invariantes de negocio al construir la entidad.

```
1 // BIEN: solo precondición técnica – llegó y tiene longitud válida
2 Guard.Against(request.Document, "Document")
3     .NotNullOrEmpty().MinLength(7).MaxLength(8);
4 // el dominio verifica que sean dígitos al construir el Value Object
```

Retorno después de ValidationErrorsAsync**ERROR COMUN**

Olvidar el `return` después de notificar los errores — el UseCase continúa ejecutando el dominio con datos inválidos.

```
1 // MAL: sin return, el dominio se ejecuta igual
2 if (errors.Count > 0)
3     await _outputPort.ValidationErrorsAsync(errors);
4 // falta return – Customer.Create() se invoca con datos inválidos
```

BUENA PRACTICA

Notificar los errores y retornar inmediatamente — el dominio no es invocado si el DTO no pasó la validación de precondiciones.

```
1 // BIEN: notifica y retorna – el dominio no es invocado
2 if (errors.Count > 0)
3 {
4     await _outputPort.ValidationErrorsAsync(errors);
5     return;
6 }
```

Registro del hub — interfaz vs. clase concreta

ERROR COMUN

Registrar el hub por su clase concreta — el UseCase queda acoplado a la implementación y no puede testearse en aislamiento.

```
1 // MAL: registrado como clase concreta
2 services.AddScoped<CreateCustomerValidatorHub>();
```

BUENA PRACTICA

Registrar la interfaz genérica con la implementación concreta — el contenedor inyectará la implementación correcta respetando el DIP.

```
1 // BIEN: interfaz → implementación concreta
2 services.AddScoped<
3     IModelValidatorHub<CreateCustomerRequest>,
4     CreateCustomerValidatorHub>();
```

Actividad

Implementá de forma completa los Guards para el caso de uso **CU-CLI-001 — Alta de Cliente** en el proyecto **CommerX.Application**, siguiendo el orden de las partes. Compilar el proyecto después de cada parte antes de continuar.

Parte A — ValidationError

1. Crear la carpeta `Common/Validation/` dentro de `CommerX.Application`.
2. Declarar el namespace: `CommerX.Application.Common.Validation`.
3. Declarar `public class ValidationError` usando primary constructor de C# 12 con parámetros `propertyName` y `message`.
4. Exponer ambos parámetros como propiedades de solo lectura con `=>`.
5. Verificar que la clase no tiene lógica, constructores adicionales ni herencia.

Parte B — GuardBuilderBase<TBuilder>

1. En la misma carpeta, crear `GuardBuilderBase.cs`.
2. Declarar la clase abstracta genérica con el constraint CRTP: `where TBuilder : GuardBuilderBase<TBuilder>`.

3. Agregar el campo `protected readonly List<ValidationError> _errors`.
4. Agregar el campo `protected readonly string _paramName`.
5. Agregar el constructor protegido que inicializa `_paramName`.
6. Exponer `Errors` como `IReadOnlyList<ValidationError>`.
7. Exponer `IsValid` como `bool` verificando `_errors.Count == 0`.
8. Agregar el método protegido `AddError(string message)`.
9. Compilar antes de continuar.

Parte C — GuardBuilderString

1. Crear `GuardBuilderString.cs` en la misma carpeta.
2. Heredar de `GuardBuilderInterface<GuardBuilderString>` — pasar el propio tipo.
3. Agregar el campo privado `string _value`.
4. Implementar el constructor que llama a `base(paramName)` e inicializa `_value`.
5. Implementar los cuatro métodos que devuelven `GuardBuilderString`:
 - a. `NotNullOrEmpty` — usa `string.IsNullOrEmpty`.
 - b. `MinLength(int min)` — compara `_value?.Length < min`.
 - c. `MaxLength(int max)` — compara `_value?.Length > max`.
 - d. `InvalidEmail` — usa `Regex` para verificar formato.
6. Verificar que cada método termina con `return this;`.
7. Compilar antes de continuar.

Parte D — Guard

1. Crear `Guard.cs` en la misma carpeta.
2. Declarar `public static class Guard`.
3. Implementar el método estático `Against(string value, string paramName)` que retorna `new GuardBuilderString(value, paramName)`.
4. Compilar antes de continuar.

Por ahora `Guard` solo tiene la sobrecarga para `string` porque todos los campos de `CreateCustomerRequest` son texto. En módulos posteriores se agregarán sobrecargas para `DateOnly` y `Guid`.

Parte E — IModelValidatorHub<TModel>

1. Crear `IModelValidatorHub.cs` en la misma carpeta.
2. Declarar la interfaz genérica `public interface IModelValidatorHub<TModel>`.
3. Declarar un único método: `IEnumerable<ValidationError> Validate(TModel model);`
4. Compilar antes de continuar.

Parte F — CreateCustomerValidatorHub

1. Crear la carpeta `Customers/Validation/` dentro de `CommerX.Application`.
2. Namespace: `CommerX.Application.Customers.Validation`.
3. Agregar `using CommerX.Application.Common.Validation;`
4. Declarar `public class CreateCustomerValidatorHub` implementando `IModelValidatorHub<CreateCustomerRequest>`.
5. Implementar el método `Validate` aplicando las reglas de la tabla: un Guard por campo, encadenando los métodos correspondientes.
6. Combinar todos los errores con `.Concat()` y retornar la secuencia.
7. Compilar antes de continuar.

Parte G — Actualizar CreateCustomerUseCase

1. Abrir `Customers/CreateCustomerUseCase.cs`.
2. Agregar `using CommerX.Application.Common.Validation;`
3. Agregar el campo privado `IModelValidatorHub<CreateCustomerRequest> _validator`.
4. Agregar el parámetro `validator` al constructor y asignarlo a `_validator`.
5. Al inicio de `ExecuteAsync`, antes del `try`, agregar:
 - a. Llamar a `_validator.Validate(request).ToList()` y guardar en `errors`.
 - b. Si `errors.Count > 0`, llamar a `_outputPort.ValidationErrorsAsync(errors)` y retornar.
6. Compilar antes de continuar.

Parte H — Actualizar DependencyContainer

1. Abrir `DependencyContainer.cs` en la raíz de `CommerX.Application`.
2. Agregar los usings: `using CommerX.Application.Common.Validation;` y `using CommerX.Application.Customers.Validation;`
3. Dentro de `AddCustomerUseCases`, después del registro del UseCase, agregar el registro del hub:

```
1 // agrega inmediatamente después del registro del UseCase
2 services.AddScoped<
3     IModelValidatorHub<CreateCustomerRequest>,
4     CreateCustomerValidatorHub>();
```

4. Compilar la solución completa — debe compilar sin errores ni warnings.
5. Verificar que el hub está registrado como `Scoped` y no como `Singleton` ni `Transient`.

Parte I — Checklist de entrega

Archivo	Ubicación	Verificación
<code>ValidationError.cs</code>	Common/Validation/	Primary constructor — dos propiedades de solo lectura
<code>GuardBuilderBase.cs</code>	Common/Validation/	Clase abstracta genérica con CRTP y where — AddError protegido
<code>GuardBuilderString.cs</code>	Common/Validation/	NotNullOrEmpty · MinLength · MaxLength · InvalidEmail encadenables
<code>Guard.cs</code>	Common/Validation/	Clase estática — método Against para string
<code>IModelValidatorHub.cs</code>	Common/Validation/	Interfaz genérica — un solo método Validate(TModel)
<code>CreateCustomerValidatorHub.cs</code>	Customers/Validation/	6 Guards — uno por campo — errores combinados con Concat
<code>CreateCustomerUseCase.cs</code>	Customers/	Injecta IModelValidatorHub — Guard antes de Customer.Create()
<code>DependencyContainer.cs</code>	raíz de Application	Registra IModelValidatorHub como Scoped

Preguntas de reflexion

1. ¿Por qué el Guard verifica que `Document` tenga entre 7 y 8 caracteres pero no verifica que sean todos dígitos? ¿Qué capa es responsable de esa regla?
2. Si `FullName` llega vacío y `Email` tiene formato incorrecto, ¿cuántos `ValidationError` devuelve el hub en total? ¿Por qué?
3. ¿Qué pasaría si quitás el `return` después de `await _outputPort.ValidationErrorsAsync(errors)` ?
4. ¿Por qué `IModelValidatorHub<T>` se registra como `Scoped` y no como `Singleton` ?
5. ¿Qué ventaja tiene que `CreateCustomerUseCase` dependa de `IModelValidatorHub<CreateCustomerRequest>` en lugar de `CreateCustomerValidatorHub` directamente?



Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Tecnicatura Superior en Desarrollo de Software | Asignatura:
Programación II | Tema: Módulo 05 — Guards — Actividad Práctica | Ciclo Lectivo 2026